



Introdução à Programação C

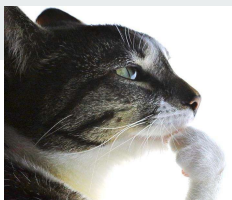
Aula 2



Introdução à Programação C

Aula 2

**MUITO OBRIGADO,
CALUMBI!!!**



Quem sou eu?

- Estudante de **Engenharia de Computação** (8º Período)
- **Monitor** de Programação C
- Membro Efetivo na **Liga Acadêmica de Hardware e Robótica (CoreTech)**
- Terminando **IC** em Astrofísica (não aguento maiskkkkk)
- Amo tudo sobre música e jogo **Futebol Americano** (sim, existe aqui)

Esse sou eu 🙌
(copiei mesmo)





Tópicos a serem mencionados:

- Funções
- Condicionais
- Switch-case
- Recursão

Antes, uma breve revisão!

Entrada e Saída

`printf()` vs `scanf()`

Entrada e Saída

`printf("a = %d", argumento);` `scanf("a = %d", &argumento);`

Tipo	char	int	float	double	char[]	(tipo)*
Máscara	%c	%d	%f	%lf	%s	%p

Vamos relembrar...

Da aula anterior...

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```



Assinatura da função "main"

A função "main" é a função principal, é por ela que seu programa começa a ser executado

A função "main" retorna um inteiro (int)

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Escopo da função "main"

Delimita o trecho de código
que pertence a função "main"

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Corpo da função "main"

É onde escrevemos o que
queremos que nosso programa
faça

C programa.c X

C programa.c > ...

```
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

Retorno da função "main"

Indica o final do programa

Retorna o valor zero para o SO
indicando que programa
encerrou sem erros

Como se chama cada parte da estrutura de um programa em C?

```
C programa.c X
C programa.c > ...
1  #include <stdio.h>
2
3  int main()
4  {
5      // Código
6      printf("Hello World\n");
7      // Código
8
9      return 0;
10 }
11
12
```

1. Cabeçalho
2. Assinatura da função main
3. Escopo da função main
4. Corpo da função main
5. Retorno da função main

Sabemos que uma função tem

1. Assinatura
2. Escopo
3. Corpo
4. Retorno

Com base nisso vamos criar uma função que recebe dois números inteiros A e B , e exibe o resultado de $A+B$, $A-B$, $A*B$, A/B e retorna $A\%B$

Sabemos que uma função tem

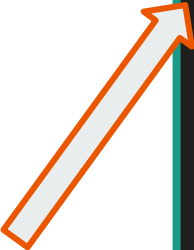
1. Assinatura

2. Escopo

3. Corpo

4. Retorno

Note que quando definimos a assinatura da função, também definimos o TIPO de dado que ela retorna



```
int minha_funcao(int a, int b)
```


Sabemos que uma função tem

1. Assinatura
2. Escopo
3. Corpo
4. Retorno

Escopo definido (mesmo que não haja nada dentro dele)

```
int minha_funcao(int a, int b)
{
}
}
```

Sabemos que uma função tem

1. Assinatura
2. Escopo
3. Corpo
4. Retorno

Nada de novo por aqui

```
int minha_funcao(int a, int b)
{
    printf("A+B = %d", a+b);
    printf("A-B = %d", a-b);
    printf("A*B = %d", a*b);
    printf("A/B = %d", a/b);
}
```

Sabemos que uma função tem

1. Assinatura
2. Escopo
3. Corpo
4. Retorno

A função irá retornar o valor de $a\%b$

```
int minha_funcao(int a, int b)
{
    printf("A+B = %d", a+b);
    printf("A-B = %d", a-b);
    printf("A*B = %d", a*b);
    printf("A/B = %d", a/b);

    return a%b;
}
```

grama.c > ...

```
#include <stdio.h>
```

```
int minha_funcao(int a, int b)
{
    printf("A+B = %d\n", a+b);
    printf("A-B = %d\n", a-b);
    printf("A*B = %d\n", a*b);
    printf("A/B = %d\n", a/b);

    return a%b;
}
```

```
int main()
{
    int retorno = minha_funcao(6,3);

    printf("retorno = %d", retorno);

    return 0;
}
```

lucasalumbi@pop-os:

A+B = 9

A-B = 3

A*B = 18

A/B = 2

retorno = 0

lucasalumbi@pop-os:

Prática:

Crie uma função que receba dois números inteiros A e B e retorne a soma deles.

Chame essa função na main e exiba o resultado.

Algumas observações sobre funções



- Em C, funções devem ser declaradas antes da função main
- Funções podem receber nenhum parâmetro
- Funções podem retornar nada (dizemos que ela retorna "void")
- Funções que recebem nada e retornam nada são chamadas de "procedimento"

ograma.c > ...

```
#include <stdio.h>
```

```
void minha_funcao()
```

```
{
```

```
    int a = 8, b = 4;
```

```
    printf("A+B = %d\n", a+b);
```

```
    printf("A-B = %d\n", a-b);
```

```
    printf("A*B = %d\n", a*b);
```

```
    printf("A/B = %d\n", a/b);
```

```
}
```

```
int main()
```

```
{
```

```
    minha_funcao();
```

```
    return 0;
```

```
}
```

Exemplo de procedimento

lucasalumbi@pop

A+B = 12

A-B = 4

A*B = 32

A/B = 2

lucasalumbi@pop

Prática:

Crie um procedimento que apenas imprime "Esse minicurso é muito bom!".

Chame-o na main.

Switch-case

Switch-Case



- Estrutura de Decisão para evitar diversos if-else aninhados;
- O valor dentro do switch é comparado com os casos descritos;
- Caso não encontre, há um resultado 'default'

Switch-Case



- Ex:

```
switch (Mes){  
    case 1: printf("janeiro \n"); break;  
    case 2: printf("fevereiro \n"); break;  
    case 3: printf("marco \n"); break;  
    case 4: printf("abril \n"); break;  
    case 5: printf("maio \n"); break;  
    case 6: printf("junho \n"); break;  
    case 7: printf("julho \n"); break;  
    case 8: printf("agosto \n"); break;  
    case 9: printf("setembro \n"); break;  
    case 10: printf("outubro \n"); break;  
    case 11: printf("novembro \n"); break;  
    case 12: printf("dezembro \n"); break;  
    default: printf("mes invalido \n");  
}  
return 0;}
```

Switch-Case



- Atenção ao **'break'**!
- Como o código é lido em sequência, caso não lembre de usá-lo, todos os casos seguintes serão executados .
- Ex: Se o Mes = 4, ele irá exibir de abril a dezembro.

Switch-Case



- Aceita char também!

Decimal	Char	Decimal	Char	Decimal	Char
60	<	70	F	80	P
61	=	71	G	81	Q
62	>	72	H	82	R
63	?	73	I	83	S
64	@	74	J	84	T
65	A	75	K	85	U
66	B	76	L	86	V
67	C	77	M	87	W
68	D	78	N	88	X
69	E	79	O	89	Y

Switch-Case



- Exercício rápido: usando switch-case, exiba os dias da semana com a mesma inicial.

Funções

Só que recursivas



Vamos analisar esse exemplo clássico

$$\text{fib}(0) = 0$$

$$\text{fib}(1) = 1$$

$$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$$

Vamos analisar esse exemplo clássico

$\text{fib}(0) = 0$

$\text{fib}(1) = 1$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Primeiro montamos a estrutura
da função

```
int fib(int n)
{
    return
}
```

Vamos analisar esse exemplo clássico

`fib(0) = 0`

`fib(1) = 1`

`fib(n) = fib(n-1) + fib(n-2)`

Depois tratamos os casos-base

```
int fib(int n)
{
    if(n == 0)
        return 0;
    if(n == 1)
        return 1;
    return
```

Vamos analisar esse exemplo clássico

$\text{fib}(0) = 0$

$\text{fib}(1) = 1$

$\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

E então definimos o retorno

```
int fib(int n)
{
    if(n == 0)
        return 0;
    if(n == 1)
        return 1;
    return fib(n-1) + fib(n-2);
}
```

Prática:

Crie uma função recursiva que receba um inteiro não negativo N, e retorne o valor do fatorial de N.

Entrada	Saída
0	1
1	1
5	120
6	720
7	5040
10	3628800
13	1932053504

Atividades para fixação:

- Crie uma FUNÇÃO que receba o raio e retorne a área do círculo
- Crie uma FUNÇÃO que receba três notas e retorne a média
- Crie uma FUNÇÃO que receba dois números inteiros A e B e retorne o maior deles
- Crie uma FUNÇÃO que recebe o peso (em kg) e a altura (em metros) e retorne o imc
- Crie uma FUNÇÃO que receba um inteiro N e retorne o valor de $N^2 + 3N + 2$

Exercício Final: Calculadora Recursiva

Calculadora Recursiva

Faça um programa que lê a entrada do usuário no formato:
“[inteiro] [operador] [inteiro]”.

Utilizando switch-case, sua calculadora deve conter as 4 operações básicas, mas deve usar recursão.

OBS: SOMA e SUBTRAÇÃO não precisam!

Atividade Para Casa:

Crie uma função recursiva que receba um inteiro positivo N e retorne o valor da soma

$$1^2+2^2+3^2+ \dots + N^2.$$

(Obs: não use fórmulas)

Entrada	Saída
1	1
2	5
9	285
15	1240
27	6930
32	11440
49	40425

Atividade Para Casa:

Crie um procedimento que leia uma hora formatada da seguinte forma:

$XhYmin$, sendo X e Y inteiros positivos

E exiba o equivalente em segundos

Entrada	Saída
0h01min	s = 60
1h00min	s = 3600
00h23min	s = 1380
9h15min	s = 33300
10h45min	s = 38700
23h59min	s = 86340
12h30min	s = 45000



Desafio — O Caçador de Números



Usando recursividade, crie um jogo interativo em que o programa deve descobrir um número secreto X (entre 1 e 1000) escolhido pelo usuário.

Funcionamento:

1. O programa tentará adivinhar o número escolhido.
2. A cada tentativa, o usuário deverá responder com:
 - '>' → se o número secreto for maior que o palpite do programa;
 - '<' → se o número secreto for menor que o palpite;
 - '=' → se o palpite estiver correto.
3. O processo se repete recursivamente até o programa acertar o número.
4. Quando o número for descoberto, exiba a mensagem:
"O numero secreto eh X" (substitua 'X' pelo valor)

Número secreto X = 46

Entrada	Saída
<	500 ?
<	250 ?
<	125 ?
<	62 ?
>	31 ?
=	46 ?
	O numero secreto eh 46

Atenção:

Use :

```
char c;  
scanf("%c", &c);  
getchar();
```

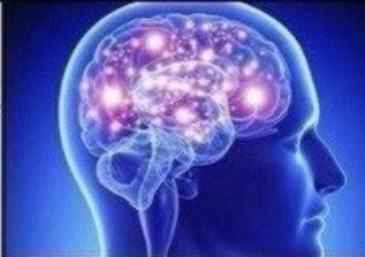
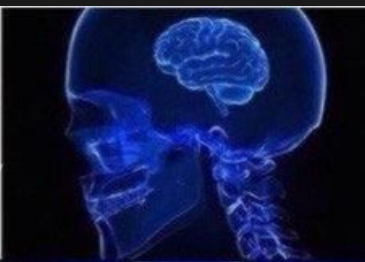
quando for ler a entrada do usuário.

Dicas:

- **Utilize recursão para repetir as tentativas.**
- **Pense em estratégias para o programa “chutar” de forma inteligente**



```
#include <stdio.h>
#include <stdlib.h>
#define _WIN32
exit(-1);
#else
int main (int argc, char *argv[])
{ printf("teste"); }
#endif
```



Obrigado pela
atenção!